

Database Server Tools Scripting Help



SQL File 3*



```
1 -- EXECUTIVE KPI --
2 -- 1.TOTAL USERS --
3
4 • CREATE VIEW TOTAL_USERS AS
5   (SELECT
6     COUNT(*) AS TOTAL_USERS
7     FROM users
8     WHERE user_id IS NOT NULL);
9
10 • SELECT * FROM TOTAL_USERS;
11
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	TOTAL_USERS
▶	2500



SQL File 3* x



```
1 -- EXECUTIVE KPI --
2 -- 2.TOTAL AI TOOLS --
3 • CREATE VIEW TOTAL_AI_TOOL AS
4   ( SELECT
5     COUNT(*) AS TOTAL_TOOLS
6     FROM tool
7     WHERE tool_id IS NOT NULL
8   );
9
10 • SELECT * FROM TOTAL_AI_TOOL;
11
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	TOTAL_TOOLS
▶	10

```
1 -- EXECUTIVE KPIs--
2 -- 3.AVERAGE DAILY USAGE PER USER IN MINUTES--
3
4 • SELECT
5     USER_ID,
6     USER_NAME,
7     ROUND(AVG(TOTAL_DURATION)) AS AVG_DAILY_USAGE
8 FROM
9     ( -- SUB QUERY--
10      SELECT
11          u.user_id USER_ID,
12          u.user_name USER_NAME,
13          s.session_date,
14          SUM(s.duration_minutes) AS TOTAL_DURATION
15      FROM sessions s
16      JOIN users u
17          ON u.user_id = s.user_id
18      GROUP BY
19          u.user_id,
20          s.session_date
21      )t
22 GROUP BY
23     USER_ID,
24     USER_NAME
25 ORDER BY USER_ID
```

SQL File 3* x



```
1 -- EXECUTIVE KPIs--
2 -- 4.TOTAL SESSIONS --
3 • CREATE VIEW TOTAL_SESSIONS AS
4 SELECT
5     COUNT(*) AS TOTAL_SESSIONS
6 FROM sessions
7 WHERE session_id IS NOT NULL;
8
9 • SELECT * FROM TOTAL_SESSIONS;
10
```

ing queries          Don't Limit     

-- EXECUTIVE KPIs--

Result Grid | Filter Rows:

Export:  | Wrap Cell Content: 

	TOTAL_PROMPT
▶	15000

SQL File 3* x



Don't Limit

```
1 -- EXECUTIVE KPIs--
2 -- 6.TOTAL TOKENS USED --
3 • SELECT
4 SUM(token) TOTAL_TOKENS
5 FROM prompt;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	TOTAL_TOKENS
▶	5398248

```
1 -- EXECUTIVE KPIs--
2 -- 7.TOTAL TOKENS USED PER AI TOOL --
3
4 -- CREATE VIEW --
5 • CREATE VIEW TOTAL_TOKENS_USED_PER_TOOL AS
6 SELECT
7 RANKS,
8 TOOL,
9 TOTAL_TOKEN_USED
10 FROM
11 -- SUB QUERY--
12 (SELECT
13     t.tool_name TOOL,
14     SUM(p.token) TOTAL_TOKEN_USED,
15     DENSE_RANK() OVER(ORDER BY SUM(p.token) DESC) RANKS
16 FROM tool t
17 JOIN sessions s
18     ON t.tool_id = s.tool_id
19 JOIN prompt p
20     ON p.session_id = s.session_id
21 GROUP BY
22     t.tool_name )t;
23
24 -- QUERY THE VIEW --
25 • SELECT * FROM TOTAL_TOKENS_USED_PER_TOOL;
```

Database Server Tools Scripting Help



SQL File 3* Administration - Performance Re...



Don't Limit

```
1 -- EXECUTIVE KPIs--
2 -- 8.AVERAGE SESSION DURATION --
3
4 • SELECT
5     TIME_FORMAT(SEC_TO_TIME(AVG(duration_minutes)*60),'%H:%i:%s') AS AVG_SESSION_DURATION
6 FROM sessions;
```

```
2      -- 9.AVERAGE TOKENS PER PROMPT --
3
4  •   CREATE VIEW avg_token_per_prompt AS
5      SELECT
6      ROUND(AVG(TOTAL_TOKEN)) AS AVERAGE_TOKEN_PER_PROMPT
7      FROM
8      (
9      SELECT
10         prompt_id,
11         SUM(token) AS TOTAL_TOKEN
12      FROM prompt
13      GROUP BY
14         prompt_id
15      )T;
16
17  •   SELECT * FROM avg_token_per_prompt;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Con

AVERAGE_TOKEN_PER_PROMPT



360



SQL File 3* Administration - Performance Re...



-- BUSSINESS INSIGHTS--

```
1
2  -- 10.MOST USED AI TOOL --
3  • CREATE VIEW MOST_USED_AI_TOOL AS
4      SELECT
5          t.tool_name AS TOOL,
6          t.company AS COMPANY,
7          COUNT(s.tool_id) USED_COUNT
8      FROM sessions s
9      JOIN tool t
10         ON t.tool_id = s.tool_id
11      GROUP BY
12         t.tool_name,
13         t.company
14      ORDER BY USED_COUNT DESC LIMIT 1;
15
16  • SELECT * FROM MOST_USED_AI_TOOL;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	TOOL	COMPANY	USED_COUNT
▶	DeepSeek	DeepSeek AI	808



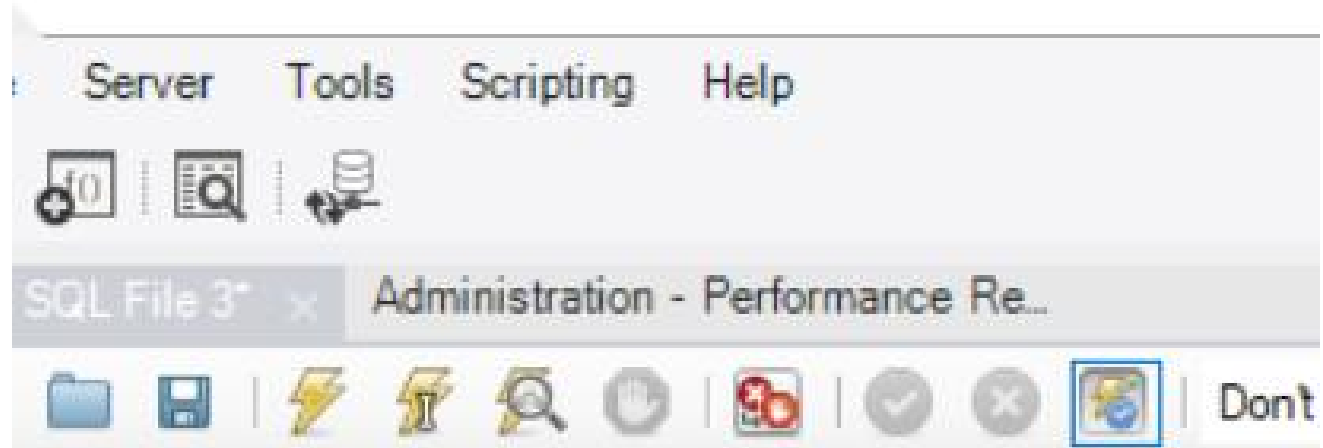
```

1                                     -- BUSSINESS INSIGHTS--
2      -- 11.LEAST USED AI TOOL --
3  • CREATE VIEW LEAST_USED_AI_TOOL AS
4      SELECT
5          t.tool_name AS TOOL,
6          t.company AS COMPANY,
7          COUNT(s.tool_id) USED_COUNT
8      FROM sessions s
9      JOIN tool t
10         ON t.tool_id = s.tool_id
11      GROUP BY
12         t.tool_name,
13         t.company
14      ORDER BY USED_COUNT ASC LIMIT 1;
15
16  • SELECT * FROM LEAST_USED_AI_TOOL;
    
```





	TOOL	COMPANY	USED_COUNT
▶	Gemini	Google	712



```
1
2      -- 12.TOTAL SESSIONS BY AI TOOL --
3
4  •   CREATE VIEW SESSIONS_BY_TOOL AS
5
6         SELECT
7         t.tool_id,
8         t.tool_name,
9         t.company,
10        COUNT(s.session_id) TOTAL_SESSION
11 FROM sessions s
12 JOIN tool t
13     ON t.tool_id = s.tool_id
14 GROUP BY
15     t.tool_id,
16     t.tool_name,
17     t.company
18 ORDER BY t.tool_id ASC;
19
20 •   SELECT * FROM SESSIONS_BY_TOOL;
```

ServerToolsScriptingHelp

SQL File 3*Administration - Performance Re...

Don't Limit

```
1 -- BUSSINESS INSIGHTS--
2 -- 13.AVERAGE SESSION DURATIONNN BY AI TOOL --
3
4 • CREATE VIEW AVERAGE_SESSION_DURATION_BY_TOOLS AS
5     SELECT
6         t.tool_id,
7         t.tool_name AS TOOL,
8         t.company COMPANY,
9         TIME_FORMAT(SEC_TO_TIME(AVG(s.duration_minutes)*60),'%H:%i:%s') AS AVERAGE_SESSION_DURATION_BY_TOOL
10    FROM sessions s
11   JOIN tool t
12      ON t.tool_id = s.tool_id
13   GROUP BY
14       t.tool_id,
15       t.tool_name,
16       t.company;
17
18 • SELECT * FROM AVERAGE_SESSION_DURATION_BY_TOOLS;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	tool_id	TOOL	COMPANY	AVERAGE_SESSION_DURATION_BY_TOOL
▶	T01	ChatGPT	OpenAI	01:22:21
	T02	Gemini	Google	01:30:07
	T03	Claude	Anthropic	01:27:45



SQL File 3* x Administration - Performance Re...



```

1                                     -- BUSSINESS INSIGHTS--
2      -- 14.MOST POPULAR PROMPT CATEGORY --
3
4  • CREATE VIEW MOST_POPULAR_PROMPT_CATEGORY AS
5  SELECT
6      category AS MOST_POPULAR_CATEGORY,
7      COUNT(*) AS TOTAL_PROMPTS
8  FROM prompt
9  WHERE prompt_id IS NOT NULL
10 GROUP BY
11     category
12 ORDER BY
13     TOTAL_PROMPTS DESC LIMIT 1;
14
15 • SELECT * FROM MOST_POPULAR_PROMPT_CATEGORY;
    
```

Result Grid



Filter Rows:

Exports:



Wrap Cell Content:



	MOST_POPULAR_CATEGORY	TOTAL_PROMPTS
▶	Coding	3758



SQL File 3* Administration - Performance Re...



```
1 -- BUSSINESS INSIGHTS--
2 -- 15.LEAST POPULAR PROMPT CATEGORY --
3
4 • CREATE VIEW LEAST_POPULAR_PROMPT_CATEGORY AS
5 SELECT
6     category AS LEAST_POPULAR_CATEGORY,
7     COUNT(*) AS TOTAL_PROMPTS
8 FROM prompt
9 WHERE prompt_id IS NOT NULL
10 GROUP BY
11     category
12 ORDER BY
13     TOTAL_PROMPTS ASC LIMIT 1;
14
15 • SELECT * FROM LEAST_POPULAR_PROMPT_CATEGORY;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	LEAST_POPULAR_CATEGORY	TOTAL_PROMPTS
▶	Image Generation	1124

SQL File 3* x Administration - Performance Re...

          Don't Limit

-- BUSSINESS INSIGHT --

```
1
2  -- 16.SESIONS BY CATEGORY --
3
4 • CREATE VIEW SESSIONS_BY_CATEGORY AS
5     SELECT
6         category AS CATEGORY,
7         COUNT(session_id) TOTAL_SESSIONS
8     FROM prompt
9     GROUP BY
10        category
11     ORDER BY
12        TOTAL_SESSIONS DESC;
13
14 • SELECT * FROM SESSIONS_BY_CATEGORY;
```

Result Grid



Filter Rows:

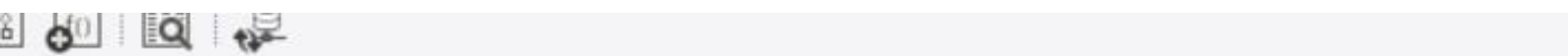
Export:



Wrap Cell Content:



	CATEGORY	TOTAL_SESSIONS
▶	Coding	3758
	Writing	3032
	Research	2657



SQL File 3* x Administration - Performance Re...



Don't Limit

```
1 -- BUSSINESS INSIGHT--
2 -- 17.TOKENS USED FOR EACH PROMPT CATEGORY --
3
4 • CREATE VIEW TOKENS_USED_FOR_EACH_PROMPT_CATEGORY AS
5     SELECT
6         category AS CATEGORY,
7         SUM(token) TOTAL_TOKENS
8     FROM prompt
9     GROUP BY
10        category
11     ORDER BY
12        TOTAL_TOKENS DESC;
13
14 • SELECT * FROM TOKENS_USED_FOR_EACH_PROMPT_CATEGORY;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	CATEGORY	TOTAL_TOKENS
▶	Coding	1778487
	Research	1098850
	Writing	1078172

SQL File 3*

Don't Limit

-- BUSSINESS INSIGHT--

-- 18.AVERAGE TOKENS USED FOR EACH PROMPT CATEGORY --

```
CREATE OR REPLACE VIEW AVG_TOKENS_USED_FOR_EACH_PROMPT_CATEGORY AS  
SELECT
```

```
    category AS CATEGORY,
```

```
    ROUND(AVG(token)) AS AVG_TOKEN
```

```
FROM
```

```
    prompt
```

```
GROUP BY
```

```
    category
```

```
ORDER BY
```

```
    AVG_TOKEN DESC;
```

```
SELECT * FROM AVG_TOKENS_USED_FOR_EACH_PROMPT_CATEGORY;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	CATEGORY	AVG_TOKEN
▶	Coding	473
	Research	414
	Writing	356

-- USER ANALYSIS--

```
1
2  -- 19.TOP 10 USERS BY ACTIVE DAYS --
3 • CREATE VIEW TOP_10_USERS_BY_ACTIVE_DAYS AS
4     SELECT
5         u.user_id AS USER_ID,
6         u.user_name AS NAME,
7         COUNT(DISTINCT DATE(s.session_date)) AS TOTAL_DAYS
8     FROM sessions s
9     JOIN users u
10        ON u.user_id = s.user_id
11    GROUP BY
12        u.user_id,
13        u.user_name
14    ORDER BY
15        TOTAL_DAYS DESC LIMIT 10;
16
17 • SELECT * FROM TOP_10_USERS_BY_ACTIVE_DAYS;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	USER_ID	NAME	TOTAL_DAYS
▶	U0270	Krishna Gupta	12
	U1898	Rahul Srivastava	9
	U1603	Mohan Agarwal	9
	U2140	Isha Tiwari	9
	U0740	Arjun Yadav	9
	U0129	Kavya Pandey	8
	U0048	Mohan Verma	8
	U0000		



```
1 -- USER ANALYSIS--
2 -- 20.TOP 10 USERS BY SESSION TIME --
3 • CREATE VIEW TOP_10_USERS_BY_SESSION_TIME AS
4     SELECT
5         u.user_id AS USER_ID,
6         u.user_name AS NAME,
7         CONCAT(
8             FLOOR(SUM(s.duration_minutes)/60), 'hour',
9             MOD(SUM(s.duration_minutes),60),'min'
10        ) AS TOTAL_SESSION_TIME
11     FROM sessions s
12     JOIN users u
13         ON u.user_id = s.user_id
14     GROUP BY
15         u.user_id,
16         u.user_name
17     ORDER BY
18         TOTAL_SESSION_TIME DESC LIMIT 10;
19
20 • SELECT * FROM TOP_10_USERS_BY_SESSION_TIME;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	USER_ID	NAME	TOTAL_SESSION_TIME
▶	U2245	Naresh Singh	9hour8min
	U0022	Meena Agarwal	9hour8min



EDA PROJECT* x project.users



```

1                                     -- USER ANALYSIS--
2      -- 21.TOP 10 USERS BY TOKEN USED --
3  •   CREATE VIEW TOP_10_USERS_BY_TOKEN_USED AS
4       SELECT
5           u.user_id AS USER_ID,
6           u.user_name AS NAME,
7           SUM(p.token) AS TOTAL_TOKEN
8       FROM prompt p
9       JOIN sessions
10          ON p.session_id = sessions.session_id
11       JOIN users u
12          ON u.user_id = sessions.user_id
13       GROUP BY
14           u.user_id,
15           u.user_name
16       ORDER BY
17           TOTAL_TOKEN DESC LIMIT 10;
18
19  •   SELECT * FROM TOP_10_USERS_BY_TOKEN_USED;
  
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	USER_ID	NAME	TOTAL_TOKEN
▶	U0270	Krishna Gupta	9762
	U0330	Priya Varma	8899
	U0627	Manoj Singh	8862
	U1603	Mohan Agarwal	8720
	U0293	Arun Sharma	8649
	U0545	G... ..	8625



Don't Limit

-- USER ANALYSIS --

```
1
2  -- 22. AVERAGE SESSIONS PER USER --
3  • CREATE VIEW AVERAGE_SESSIONS_PER_USER AS
4  SELECT
5      USER_ID,
6      NAME,
7      ROUND(AVG(TOTAL_SESSIONS),2) AS AVERAGE_SESSIONS_PER_USER
8  FROM
9      -- SUB QUERY--
10     (SELECT
11         u.user_id AS USER_ID,
12         u.user_name AS NAME,
13         COUNT(s.session_id) AS TOTAL_SESSIONS
14     FROM sessions s
15     JOIN users u
16         ON u.user_id = s.user_id
17     GROUP BY
18         u.user_id,
19         u.user_name
20     )t
21 GROUP BY
22     USER_ID, NAME,
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Fetch rows

	USER_ID	NAME	AVERAGE_SESSIONS_PER_USER
▶	U0001	Aisha Malhotra	2.00
	U0002	Divya Agarwal	1.00
	U0003	Pooja Sinha	4.00



Don't Limit

-- USER ANALYSIS --

23 -- AVERAGE TOKENS PER USER --

CREATE VIEW AVERAGE_TOKENS_PER_USER AS

SELECT DISTINCT

u.user_id AS USER_ID,

u.user_name AS NAME,

SUM(p.token) OVER(PARTITION BY u.user_id) TOTAL_TOKENS,

ROUND(AVG(p.token) OVER(PARTITION BY u.user_id)) AVG_TOKENS

FROM prompt p

JOIN sessions s

ON s.session_id = p.session_id

JOIN users u

ON u.user_id = s.user_id;

SELECT * FROM AVERAGE_TOKENS_PER_USER;

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Fetch rows

	USER_ID	NAME	TOTAL_TOKENS	AVG_TOKENS
▶	U0001	Aisha Malhotra	3028	303
	U0002	Divya Agarwal	967	322
	U0003	Pooja Sinha	3214	292

-- USER ANALYSIS --

-- 24. SINGLE AND MULTIPLE AI TOOL USERS --

SELECT

u.user_id USER_ID,

u.user_name USER_NAME,

COUNT(DISTINCT(t.tool_id)) TOOL_COUNT,

CASE

WHEN COUNT(DISTINCT(t.tool_id)) = 1 THEN 'SINGLE TOOL USER'

WHEN COUNT(DISTINCT(t.tool_id)) >= 2 THEN 'MULTIPLE TOOL USER'

WHEN COUNT(DISTINCT(t.tool_id)) = 0 THEN '0'

ELSE 'NONE'

END AS USER_CATEGORY

FROM tool t

JOIN sessions s ON s.tool_id = t.tool_id

JOIN users u ON u.user_id = s.user_id

GROUP BY

u.user_id,u.user_name

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Fetch rows:

	USER_ID	USER_NAME	TOOL_COUNT	USER_CATEGORY
▶	U0001	Aisha Malhotra	2	MULTIPLE TOOL USER
	U0002	Divya Agarwal	1	SINGLE TOOL USER
	U0003	Pooja Sinha	4	MULTIPLE TOOL USER
	U0004	Ashok Tiwari	1	SINGLE TOOL USER

```
1 -- USER ANALYSIS --
2 -- 25. MOST ACTIVE PROFESSION --
3
4 • SELECT
5     u.profession AS MOST_ACTIVE_PROFESSION,
6     COUNT(p.prompt_id) AS TOTAL_PROMPT
7 FROM prompt p
8 JOIN sessions s
9     ON s.session_id = p.session_id
10 JOIN users u
11     ON u.user_id = s.user_id
12 GROUP BY
13     u.profession
14 ORDER BY
15     TOTAL_PROMPT DESC LIMIT 1
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	MOST_ACTIVE_PROFESSION	TOTAL_PROMPT
▶	Legal Advisor	420

```
1 -- USER ANALYSIS --
2 -- 26. MOST ACTIVE CITY --
3
4 • SELECT
5     u.city AS MOST_ACTIVE_CITY,
6     COUNT(u.user_id) TOTAL_USERS,
7     COUNT(p.prompt_id) AS TOTAL_PROMPT,
8     COUNT(s.session_id) AS TOTAL_SESSIONS
9 FROM prompt p
10 JOIN sessions s
11     ON s.session_id = p.session_id
12 JOIN users u
13     ON u.user_id = s.user_id
14 GROUP BY
15     u.city
16 ORDER BY
17     TOTAL_PROMPT AND TOTAL_SESSIONS DESC LIMIT 1;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	MOST_ACTIVE_CITY	TOTAL_USERS	TOTAL_PROMPT	TOTAL_SESSIONS
▶	Vadodara	757	757	757

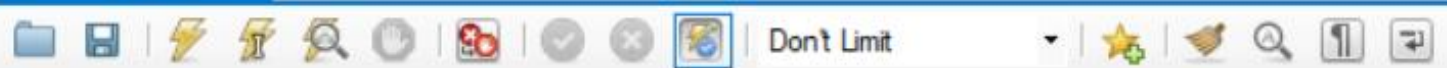
Server Tools Scripting Help



EDA PROJECT* x



```
1      -- TOP 10 PROFESSIONS BY SESSION COUNT--
2  •   SELECT
3      u.profession AS PROFESSION,
4      COUNT(DISTINCT s.session_id) AS TOTAL_SESSIONS
5  FROM sessions s
6  JOIN users u
7      ON u.user_id = s.user_id
8  GROUP BY
9      u.profession
10 ORDER BY
11     TOTAL_SESSIONS DESC LIMIT 10
12
```



Don't Limit

```
1 -- PROFESSION & CITY INSIGHTS --
2 -- 28.TOKENS BY PROFESSION --
3
4 • SELECT
5     u.profession AS PEOFESSION,
6     SUM(p.token) TOKEN_USED
7 FROM prompt p
8 LEFT JOIN sessions s
9     ON s.session_id = p.session_id
10 JOIN users u
11     ON u.user_id = s.user_id
12 GROUP BY
13     u.profession
14 ORDER BY
15     TOKEN_USED DESC
16
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	PEOFESSION	TOKEN_USED
▶	QA Engineer	148092
	Legal Advisor	144944
	Content Writer	141918
	Accountant	141690

EDA PROJECT*

users

         Don't Limit

```
1                                     -- PROFESSION & CITY INSIGHTS --
2      -- 29.AVG SESSION DURATION BY PROFESSION --
3
4  •  SELECT
5      u.profession PROFESSION,
6      CONCAT(
7          FLOOR(AVG(s.duration_minutes) / 60), 'hour',
8          FLOOR(AVG(s.duration_minutes) % 60), 'Min'
9      ) AS AVERAGE_DURATION
10 FROM sessions s
11 JOIN users u
12     ON u.user_id = s.user_id
13 GROUP BY
14     u.profession
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	PROFESSION	AVERAGE_DURATION
▶	Systems Administrator	1hour15Min
	Customer Success Manager	1hour34Min
	Accountant	1hour28Min



```
1 -- PROFESSION & CITY INSIGHTS --
2 -- 30.FAVORITE AI TOOL BY PROFESSION--
3 • SELECT
4     PROFESSION,
5     FAVORITE_TOOL
6 FROM
7     (SELECT
8         u.profession PROFESSION,
9         t.tool_name AS FAVORITE_TOOL,
10        COUNT(s.session_id) TOTAL_SESSIONS,
11        DENSE_RANK() OVER(PARTITION BY u.profession ORDER BY COUNT(s.session_id) DESC ) AS TOOL_RANK
12 FROM tool t
13 JOIN sessions s
14     ON s.tool_id = t.tool_id
15 JOIN users u
16     ON u.user_id = s.user_id
17 GROUP BY
18     u.profession,
19     t.tool_name) t
20 WHERE TOOL_RANK = 1; -- CONDITITON --
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	PROFESSION	FAVORITE_TOOL
▶	Accountant	Perplexity
	Backend Developer	Midjourney
	Business Analyst	Copilot

-- PROFESSION & CITY INSIGHTS --

```
1
2  -- 31.SESIONS BY CITY --
3 • SELECT
4     u.city AS CITY,
5     COUNT(DISTINCT(s.session_id))AS TOTAL_SESSIONS
6 FROM sessions s
7 JOIN users u
8     ON u.user_id = s.user_id
9 GROUP BY
10     u.city
11
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	CITY	TOTAL_SESSIONS
▶	Agra	184
	Ahmedabad	211
	Aurangabad	198
	Bangalore	235



EDA PROJECT* x users



Don't Limit

-- PROFESSION & CITY INSIGHTS --

```
1
2  -- 32.TOKEN BY CITY --
3  • CREATE VIEW TOKEN_BY_CITY AS
4  SELECT
5      u.city AS CITY,
6      SUM(p.token) AS TOTAL_TOKENS_USED
7  FROM prompt p
8  JOIN sessions s
9      ON s.session_id = p.session_id
10 JOIN users u
11     ON u.user_id = s.user_id
12 GROUP BY
13     u.city;
14
```



```
1 -- PROFESSION & CITY INSIGHTS --
2 -- 33. FAVORITE AI TOOL BY CITY --
3 • WITH CTE AS
4 (
5     SELECT
6         u.city AS CITY,
7         t.tool_name AS FAVORITE_TOOL,
8         COUNT(s.session_id) AS TOTAL_SESSION,
9         SUM(s.duration_minutes) AS TOTAL_MIN,
10        COUNT(DISTINCT(s.session_date)) AS TOTAL_DAYS_USED,
11        ROW_NUMBER() OVER(
12            PARTITION BY u.city
13            ORDER BY
14                COUNT(s.session_id) DESC,
15                SUM(s.duration_minutes) DESC,
16                COUNT(DISTINCT(s.session_date)) DESC
17        ) AS PRIMARY_RANK
18    FROM sessions s
19    JOIN tool t
20        ON t.tool_id = s.tool_id
21    JOIN users u
22        ON u.user_id = s.user_id
23    GROUP BY
24        u.city,
25        t.tool_name
26 )
27 SELECT CITY, FAVORITE_TOOL FROM CTE WHERE PRIMARY_RANK = 1;
```



EDA PROJECT* x



Don't Limit

```
1 -- TIME TRENDS --
2 -- 34.MONTHLY ACTIVE USERS AND OVERALL AVG USERS --
3 • WITH MONTHLY_ACTIVE_USERS AS
4 (
5     SELECT
6         DATE_FORMAT(s.session_date, '%b-%Y') AS MON_YEAR,
7         YEAR(s.session_date) AS YEAR,
8         MONTH(s.session_date) AS MON,
9         COUNT(DISTINCT(u.user_id)) AS TOTAL_USERS
10    FROM users u
11   JOIN sessions s
12      ON s.user_id = u.user_id
13   GROUP BY
14       YEAR(s.session_date),
15       MONTH(s.session_date),
16       DATE_FORMAT(s.session_date, '%b-%Y')
17 )
18 SELECT
19     MON_YEAR,
20     TOTAL_USERS,
21     ROUND(AVG(TOTAL_USERS) OVER()) AS OVERALL_AVG_USERS FROM MONTHLY_ACTIVE_USERS;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	MON_YEAR	TOTAL_USERS	OVERALL_AVG_USERS
	Dec-2024	156	154
	Jan-2025	146	154
	Feb-2025	146	154
	Mar-2025	160	154
	Apr-2025	175	154

EDA PROJECT ×

Don't Limit



```
1 -- TIME TRENDS --
2 -- 35.MONTHLY AND OVERALL AVG SESSIONS --
3 • WITH CTE AS
4   (SELECT
5     DATE_FORMAT(session_date, '%b-%Y') AS MON_YEAR,
6     YEAR(session_date) AS YEAR,
7     MONTH(session_date) AS MONTH,
8     COUNT(DISTINCT(session_id)) AS TOTAL_SESSION
9   FROM sessions
10  GROUP BY
11    MON_YEAR,
12    YEAR,
13    MONTH)
14 SELECT
15   MON_YEAR,
16   TOTAL_SESSION,
17   ROUND(AVG(TOTAL_SESSION) OVER()) AS OVERALL_AVG_SESSIONS
18 FROM CTE
19 ORDER BY YEAR, MONTH;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	MON_YEAR	TOTAL_SESSION	OVERALL_AVG_SESSIONS
	Nov-2024	138	174
	Dec-2024	170	174
	Jan-2025	155	174
	Feb-2025	151	174
	Mar-2025	172	174
	Apr-2025	185	174

EDA PROJECT

Don't Limit

```
1 -- TIME TRENDS --
2 -- 36.MONTHLY AND OVERALL AVG TOKENS USED --
3 • WITH CTE AS
4     (SELECT
5         DATE_FORMAT(s.session_date, '%b-%Y') AS MON_YEAR,
6         YEAR(s.session_date) AS YEAR,
7         MONTH(s.session_date) AS MONTH,
8         SUM(p.token) AS TOTAL_TOKEN_USED
9     FROM sessions s
10    JOIN prompt p
11     ON p.session_id = s.session_id
12    GROUP BY
13         MON_YEAR,
14         YEAR,
15         MONTH)
16 SELECT
17     MON_YEAR,
18     TOTAL_TOKEN_USED,
19     ROUND(AVG(TOTAL_TOKEN_USED) OVER()) AS OVERALL_AVG_TOKEN_USED
20 FROM CTE ORDER BY YEAR, MONTH;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	MON_YEAR	TOTAL_TOKEN_USED	OVERALL_AVG_TOKEN_USED
	Jul-2023	21149	125541
	Aug-2023	23383	125541
	Sep-2023	33572	125541
	Oct-2023	42130	125541
	Nov-2023	38083	125541
	Dec-2023	45046	125541
	Jan-2024	42103	125541

```
2  -- 37.TOTAL,AVG,OVERALL AVG SESSION DURATION BY MONTH --
3  • WITH CTE AS
4      (SELECT
5          DATE_FORMAT(MIN(session_date),'%b-%Y') AS MON_YEAR,
6          YEAR(session_date) AS YEAR,
7          MONTH(session_date) AS MONTH,
8          SUM(duration_minutes) AS TOTAL_DURATION,
9          ROUND(AVG(duration_minutes)) AS AVG_DURATION
10     FROM sessions
11     GROUP BY
12         YEAR(session_date),
13         MONTH(session_date)
14     ORDER BY
15         YEAR,
16         MONTH)
17  SELECT
18     MON_YEAR,
19     CONCAT(
20         FLOOR(TOTAL_DURATION / 60),'hour',
21         MOD(TOTAL_DURATION,60),'min'
22     ) AS TOTAL_DURATION,
23     CONCAT(
24         FLOOR(AVG_DURATION / 60),'hour',
25         MOD(ROUND(AVG_DURATION),60),'min'
26     ) AS AVG_DURATION,
27     CONCAT(
28         FLOOR(AVG(AVG_DURATION) OVER() / 60),'hour',
29         MOD(ROUND(AVG(AVG_DURATION) OVER()),60),'min') AS OVERALL_AVG_DURATION FROM CTE ORDER BY YEAR,MONTH;
```

```
1 -- TIME TRENDS --
2 -- 38.DAILY USAGE TREND --
3 • SELECT
4     DATE_FORMAT(s.session_date, '%d-%b-%Y') AS USAGE_DATE,
5     COUNT(DISTINCT(u.user_id)) AS TOTAL_USERS,
6     COUNT(DISTINCT(t.tool_id)) AS TOTAL_TOOL_USED,
7     COUNT(DISTINCT(p.prompt_id)) AS TOTAL_PROMPTS,
8     SUM(p.token) AS TOTAL_TOKEN_USED,
9     COUNT(DISTINCT(s.session_id)) AS TOTAL_SESSIONS,
10    CONCAT(
11        FLOOR(SUM(s.duration_minutes) / 60), 'hour',
12        MOD(SUM(s.duration_minutes), 60), 'min'
13    ) AS TOTAL_DURATION
14 FROM
15     sessions s
16 JOIN prompt p
17     ON p.session_id = s.session_id
18 JOIN tool t
19     ON t.tool_id = s.tool_id
20 JOIN users u
21     ON u.user_id = s.user_id
22 GROUP BY
23     s.session_date
24 ORDER BY
25     s.session_date;
```

... -- TIME TRENDS -- Untitled-1 •

```
1      -- TIME TRENDS --
2      -- 39.TOP 3 PEAK USAGE MONTH --
3      WITH PEAK_USAGE_MONTH AS
4      (SELECT
5          DATE_FORMAT(s.session_date,'%b-%Y') AS MONTH,
6          COUNT(DISTINCT(s.user_id)) AS TOTAL_USERS,
7          COUNT(DISTINCT(s.tool_id)) AS TOTAL_TOOLS_USED,
8          COUNT(DISTINCT(p.prompt_id)) AS TOTAL_PROMPTS,
9          SUM(p.token) AS TOTAL_TOKEN_USED,
10         COUNT(DISTINCT(s.session_id)) AS TOTAL_SESSIONS,
11         CONCAT(
12             FLOOR(SUM(s.duration_minutes) / 60),'hour',
13             MOD(SUM(s.duration_minutes),60),'min'
14         ) AS TOTAL_DURATION,
15         DENSE_RANK() OVER(ORDER BY COUNT(u.user_id) DESC ) AS USERS_RANK,
16         ROW_NUMBER() OVER(ORDER BY COUNT(DISTINCT(t.tool_id))DESC) AS TOOL_RANK,
17         DENSE_RANK() OVER(ORDER BY COUNT(DISTINCT(p.prompt_id) )DESC) AS PROMPT_RANK,
18         DENSE_RANK() OVER(ORDER BY SUM(p.token) DESC) AS TOKEN_RANK,
19         DENSE_RANK() OVER(ORDER BY COUNT(DISTINCT(s.session_id))DESC) AS SESSION_RANK,
20         DENSE_RANK() OVER(ORDER BY SUM(s.duration_minutes) DESC) AS DURATION_RANK
21     FROM sessions s
22     JOIN prompt p
23         ON p.session_id = s.session_id
24     JOIN tool t
25         ON t.tool_id = s.tool_id
26     JOIN users u
27         ON u.user_id = s.user_id
28     GROUP BY
29         DATE_FORMAT(s.session_date,'%b-%Y'))
30 SELECT
31     ROW_NUMBER() OVER(ORDER BY TOTAL_USERS DESC) AS PEAK_RANK,
32     MONTH,
33     TOTAL_USERS,
34     TOTAL_TOOLS_USED,
35     TOTAL_PROMPTS,
36     TOTAL_TOKEN_USED,
37     TOTAL_SESSIONS,
38     TOTAL_DURATION
39 FROM PEAK_USAGE_MONTH
40 WHERE LEAST(USERS_RANK,TOOL_RANK,PROMPT_RANK,TOKEN_RANK,SESSION_RANK,DURATION_RANK) <=3;
```



EDA PROJECT × users



```
1 -- ADVANCED BONUS INSIGHTS --
2 -- 40.TOOL USAGE PERCENTAGE--
3 • SELECT
4     t.tool_id AS TOOL_ID,
5     t.tool_name AS TOOL_NAME,
6     COUNT(s.session_id) TOTAL_SESSIONS,
7     ROUND
8     (COUNT(s.session_id) * 100.0 /
9     (SELECT COUNT(*) FROM sessions),2) AS USE_PERCENTAGE
10 FROM sessions s
11 JOIN tool t
12     ON t.tool_id = s.tool_id
13 GROUP BY
14     t.tool_id,
15     t.tool_name;
```

Result Grid Filter Rows: Export: Wrap Cell Content:

	TOOL_ID	TOOL_NAME	TOTAL_SESSIONS	USE_PERCENTAGE
▶	T01	ChatGPT	723	9.64
	T02	Gemini	712	9.49
	T03	Claude	731	9.75
	T04	Copilot	763	10.17
	T05	Perplexity	760	10.13



EDA PROJECT* x users



```
1 -- ADVANCED BONUS INSIGHTS --
2 -- 41.PROMPT CATEGORY USAGE PERCENTAGE--
3 • SELECT
4     p.category AS CATEGORY,
5     COUNT(s.session_id) AS TOTAL_SESSIONS,
6     ROUND(
7         COUNT(s.session_id) * 100.0 /
8         (SELECT COUNT(*) FROM sessions),2) AS PERCENTAGE
9 FROM sessions s
10 JOIN prompt p
11     ON p.session_id = s.session_id
12 GROUP BY
13     p.category
14 ORDER BY
15     PERCENTAGE DESC
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	CATEGORY	TOTAL_SESSIONS	PERCENTAGE
▶	Coding	3758	50.11
	Writing	3032	40.43
	Research	2657	35.43
	Business	2242	29.89
	Learning	2187	29.16
	Image Generation	1124	14.99


```
1 -- ADVANCED BONUS INSIGHTS --
2 -- 43.AVERAGE TOKENS PER SESSION--
3 • SELECT
4     session_id AS SESSION_ID,
5     ROUND(AVG(token),2) AS AVG_TOKEN_USED
6 FROM prompt
7 GROUP BY session id
```

Result Grid   Filter Rows:  Export:  Wrap Cell Content:  Fetch rows:

	SESSION_ID	AVG_TOKEN_USED
▶	S0001	471.50
	S0002	550.00
	S0003	182.50
	S0004	226.33
	S0005	482.00
	S0006	415.25
	S0007	587.50
	S0008	335.80
	S0009	470.00
	S0010	174.00
	S0011	256.50



EDA PROJECT* x users



```
1 -- ADVANCED BONUS INSIGHTS --
2 -- 44.TOOL X CATEGORY MATRIX--
3 • SELECT
4     t.tool_name AS TOOL_NAME,
5     p.category AS CATEGORY,
6     COUNT(DISTINCT(p.prompt_id)) AS TOTAL_PROMPTS,
7     COUNT(DISTINCT(s.session_id)) AS TOTAL_SESSIONS,
8     SUM(p.token) AS TOTAL_TOKEN_USED,
9     COUNT(DISTINCT(s.session_date)) AS TOTAL_DAYS
10 FROM prompt p
11 JOIN sessions s
12     ON s.session_id = p.session_id
13 JOIN tool t
14     ON t.tool_id = s.tool_id
15 GROUP BY t.tool_name, p.category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	TOOL_NAME	CATEGORY	TOTAL_PROMPTS	TOTAL_SESSIONS	TOTAL_TOKEN_USED	TOTAL_DAYS
▶	Canva AI	Business	228	195	73527	170
	Canva AI	Coding	377	303	181968	246
	Canva AI	Image Generation	111	103	19111	95
	Canva AI	Learning	218	188	51061	169
	Canva AI	Research	246	212	99356	188
	Canva AI	Writing	329	274	117975	224
	ChatGPT	Business	221	187	73926	163
	ChatGPT	Coding	402	305	193564	251
	ChatGPT	Image Generation	115	109	18730	102



EDA PROJECT* x users



Don't Limit

```

1                                     -- ADVANCED BONUS INSIGHTS --
2      -- 45.PROFESSION X TOOL MATRIX--
3      • SELECT
4          t.tool_name AS TOOL_NAME,
5          u.profession AS PROFESSION,
6          COUNT(DISTINCT(u.user_id)) AS TOTAL_USERS,
7          COUNT(DISTINCT(p.prompt_id)) AS TOTAL_PROMPTS,
8          COUNT(DISTINCT(s.session_id)) AS TOTAL_SESSIONS,
9          SUM(p.token) AS TOTAL_TOKEN_USED,
10         COUNT(DISTINCT(s.session_date)) AS TOTAL_DAYS
11 FROM prompt p
12 JOIN sessions s
13     ON s.session_id = p.session_id
14 JOIN users u
15     ON u.user_id = s.user_id
16 JOIN tool t
17     ON t.tool_id = s.tool_id
18 GROUP BY t.tool_name, u.profession;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	TOOL_NAME	PROFESSION	TOTAL_USERS	TOTAL_PROMPTS	TOTAL_SESSIONS	TOTAL_TOKEN_USED	TOTAL_DAYS
▶	Canva AI	Accountant	9	18	9	5413	9
	Canva AI	Backend Developer	12	28	14	8702	14
	Canva AI	Business Analyst	13	18	13	7402	13
	Canva AI	Business Development Manager	16	38	18	12608	18
	Canva AI	CIO	21	57	24	20486	24
	Canva AI	Cloud Architect	10	26	10	9348	10

```

1
2  -- 46.CITY X TOOL MATRIX--
3 •  SELECT
4      t.tool_name AS TOOL_NAME,
5      u.city AS CITY,
6      COUNT(DISTINCT(u.user_id)) AS TOTAL_USERS,
7      COUNT(DISTINCT(p.prompt_id)) AS TOTAL_PROMPTS,
8      COUNT(DISTINCT(s.session_id)) AS TOTAL_SESSIONS,
9      SUM(p.token) AS TOTAL_TOKEN_USED,
10     COUNT(DISTINCT(s.session_date)) AS TOTAL_DAYS
11 FROM prompt p
12 JOIN sessions s
13     ON s.session_id = p.session_id
14 JOIN users u
15     ON u.user_id = s.user_id
16 JOIN tool t
17     ON t.tool_id = s.tool_id
18 GROUP BY t.tool_name, u.city;

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	TOOL_NAME	CITY	TOTAL_USERS	TOTAL_PROMPTS	TOTAL_SESSIONS	TOTAL_TOKEN_USED	TOTAL_DAYS
▶	Canva AI	Agra	15	32	16	12416	16
	Canva AI	Ahmedabad	19	43	19	13376	19
	Canva AI	Aurangabad	15	29	15	11445	14
	Canva AI	Bangalore	22	49	24	18381	24
	Canva AI	Calicut	17	47	21	17809	21
	Canva AI	Chandigarh	19	55	24	17897	24
	Canva AI	Chennai	11	32	12	10507	12
	Canva AI	Coimbatore	16	44	21	15120	21
	Canva AI	Delhi	17	45	18	16807	18

Result 8 x

Output

-- TIME TRENDS -- Untitled-1 •

-- ADVANCED BONUS INSIGHTS -- Untitled-2 •

Untitled...

open

+2'

```

1      -- ADVANCED BONUS INSIGHTS --
2      -- 47.TOP 100 ENGAGED USERS --
3      WITH CTE AS
4      (
5          SELECT
6              u.user_id AS USER_ID,
7              u.user_name AS USER_NAME,
8              u.city AS CITY,
9              COUNT(DISTINCT(t.tool_id)) AS TOTAL_TOOLS_USED,
10             COUNT(DISTINCT(p.prompt_id)) AS TOTAL_PROMPT,
11             COUNT(DISTINCT(s.session_id)) AS TOTAL_SESSION,
12             SUM(p.token) AS TOTAL_TOKEN_USED,
13             COUNT(DISTINCT(s.session_date)) AS TOTAL_DAYS,
14             CASE
15                 WHEN FLOOR(SUM(s.duration_minutes) / 60) = 0
16                 THEN
17                     CONCAT(
18                         MOD(SUM(s.duration_minutes),60),' MINUTES'
19                     )
20                 WHEN MOD(SUM(s.duration_minutes),60) = 0
21                 THEN
22                     CONCAT(
23                         FLOOR(SUM(s.duration_minutes) / 60),' HOURS'
24                     )
25                 ELSE
26                     CONCAT(
27                         FLOOR(SUM(s.duration_minutes) / 60),' Hour',
28                         MOD(SUM(s.duration_minutes),60),' Min'
29                     )
30                 END AS TOTAL_DURATION,
31             DENSE_RANK() OVER(ORDER BY COUNT(DISTINCT(t.tool_id))DESC )AS TOOL_RANK,
32             DENSE_RANK() OVER(ORDER BY COUNT(DISTINCT(p.prompt_id))DESC )AS PROMPT_RANK,
33             DENSE_RANK() OVER(ORDER BY COUNT(DISTINCT(s.session_id))DESC )AS SESSION_RANK,
34             DENSE_RANK() OVER(ORDER BY SUM(p.token) DESC )AS TOKEN_RANK,
35             DENSE_RANK() OVER(ORDER BY COUNT(DISTINCT(s.session_date))DESC )AS DAYS_RANK,
36             DENSE_RANK() OVER(ORDER BY SUM(s.duration_minutes)DESC)AS DURATION_RANK
37         FROM sessions s
38         JOIN prompt p
39             ON s.session_id = p.session_id
40         JOIN users u
41             ON u.user_id = s.user_id
42         JOIN tool t
43             ON t.tool_id = s.tool_id
44         GROUP BY u.user_id, u.user_name, u.city
45     ),
46     OVERALL AS
47     (
48         SELECT *,
49             (TOOL_RANK +
50             PROMPT_RANK +
51             SESSION_RANK +
52             TOKEN_RANK +
53             DAYS_RANK +
54             DURATION_RANK) AS SCORE
55     FROM CTE
56     ),
57     FINAL_RANK AS
58     (
59         SELECT *,
60             ROW_NUMBER() OVER (ORDER BY SCORE ASC) AS OVERALL_RANK
61     FROM OVERALL
62     )
63     SELECT
64         OVERALL_RANK,
65         USER_ID,
66         USER_NAME,
67         CITY,
68         TOTAL_TOOLS_USED,
69         TOTAL_SESSION,
70         TOTAL_PROMPT,
71         TOTAL_TOKEN_USED,
72         TOTAL_DAYS,
73         TOTAL_DURATION
74     FROM FINAL_RANK
75     WHERE OVERALL_RANK <= 100;

```

-- ADVANCED BONUS INSIGHTS -- Untitled-3

-- ADVANCED BO

```
1
2 -- 48.HEAVY USERS vs LIGHT USERS --
3 CREATE VIEW HEAVY_USER_LIGHT_USER AS
4 WITH CTE AS
5 (
6     SELECT
7         u.user_id AS USER_ID,
8         u.user_name AS USER_NAME,
9         COUNT(DISTINCT(s.session_id)) AS TOTAL_SESSIONS,
10        COUNT(DISTINCT(p.prompt_id)) AS TOTAL_PROMPTS,
11        SUM(p.token) AS TOTAL_TOKEN_USED
12    FROM prompt p
13    JOIN sessions s
14        ON s.session_id = p.session_id
15    JOIN users u
16        ON u.user_id = s.user_id
17    GROUP BY
18        u.user_id,
19        u.user_name
20 ),
21 USER_SCORE AS
22 (
23     SELECT *,
24     -- Prompt Score --
25     CASE
26         WHEN TOTAL_PROMPTS >=20 THEN 3
27         WHEN TOTAL_PROMPTS BETWEEN 10 AND 19 THEN 2
28         ELSE 1
29     END +
30     -- Session Score --
31     CASE
32         WHEN TOTAL_SESSIONS >= 9 THEN 3
33         WHEN TOTAL_SESSIONS >= 5 THEN 2
34         ELSE 1
35     END +
36     -- Token Score --
37     CASE
38         WHEN TOTAL_TOKEN_USED >= 7000 THEN 3
39         WHEN TOTAL_TOKEN_USED BETWEEN 3500 AND 6999 THEN 2
40         ELSE 1
41     END AS TOTAL_SCORE
42    FROM CTE
43 )
44 SELECT *,
45 CASE
46     WHEN TOTAL_SCORE >= 8 THEN 'HEAVY USER'
47     WHEN TOTAL_SCORE BETWEEN 5 AND 7 THEN 'MEDIUM USER'
48     ELSE 'LIGHT USER'
49 END AS USER_CATEGORY
50 FROM USER_SCORE
51 ORDER BY TOTAL_SCORE DESC;
52
53
```

EDA PROJECT* users project.tool

Don't Limit

```
1  -- 49.LONGEST SESSION --
2  • SELECT
3  session_id AS SESSION_ID,
4  CASE
5  WHEN FLOOR(duration_minutes /60)= 0
6  THEN
7      CONCAT(
8          MOD(duration_minutes ,60), ' Minutes')
9  WHEN MOD(duration_minutes ,60) = 0
10 THEN
11     CONCAT(
12         FLOOR(duration_minutes /60), ' Hour')
13 ELSE
14     CONCAT(
15         FLOOR(duration_minutes /60),' Hour',
16         MOD(duration_minutes ,60),' Min'
17     ) END AS LONGEST_SESSION
18 FROM sessions -- IF ONE OR MORE USERS HAVE SAME LONGEST_DURATION USE(session_id in ASC order) --
19 ORDER BY duration_minutes DESC , session_id ASC LIMIT 1; -- TO GET THE OLDEST SESSION (THE FIRST ONE CREATED) --
```

Result Grid

Filter Rows:

Exports: Wrap Cell Content:

SESSION_ID	LONGEST_SESSION
S0542	8 Hour

Result Grid

EDA PROJECT* users project tool
ate a new stored procedure in the active schema in the connected server
Don't Limit

1 -- ADVANCED BONUS INSIGHTS --
2 -- 50.SHORTEST SESSION --
3 • SELECT
4 session_id AS SESSION_ID,
5 CASE
6 WHEN FLOOR(duration_minutes / 60) = 0
7 THEN
8 CONCAT(
9 MOD(duration_minutes , 60), ' Minutes')
10 WHEN MOD(duration_minutes , 60) = 0
11 THEN
12 CONCAT(
13 FLOOR(duration_minutes / 60), ' Hour')
14 ELSE
15 CONCAT(
16 FLOOR(duration_minutes / 60), ' Hour',
17 MOD(duration_minutes , 60), ' Min'
18) END AS SHORTEST_DURATION
19 FROM sessions -- IF ONE OR MORE USERS HAVE SAME SHORTEST_DURATION USE(session_id in ASC order) --
20 ORDER BY duration_minutes ASC , session_id ASC LIMIT 1; -- TO GET THE OLDEST SESSION (THE FIRST ONE CREATED) --

Result Grid Filter Rows: Export: Wrap Cell Content:

	SESSION_ID	SHORTEST_DURATION
▶	S0039	5 Minutes

Result Grid